**DESIGN AND ANALYSIS OF ALGORITHMS**

Assignment 2: GUI-Based Algorithm Implementation

# Smart ATM Cash Dispenser

*Minimum Coin / Note Change using Dynamic Programming*

| Item | Details |
|---|---|
| Student | Varun P Das |
| Roll No. | 29 |
| Admission No. | AJC24CC029 |
| Algorithm | Minimum Coin Change (Dynamic Programming) |
| Language / GUI | Python 3, Tkinter |
| Source files | coin_core.py, atm_gui.py, test_coin.py |

## 1. Problem Statement

An ATM stores banknotes of several denominations, for example 500, 200, 100, 50 and 10 rupees. When a customer withdraws an amount, the machine should pay it using the fewest notes possible. This keeps dispensing quick and reduces cash handling.

A common shortcut is the greedy method: always hand out the largest note that still fits. It is fast, but for some denomination sets it uses more notes than necessary, or even fails to pay an amount that can be paid. This project solves the problem with dynamic programming, which is always optimal, and displays the table being filled step by step. It also compares the result with the greedy method so the difference is visible.

### Objectives

- Apply dynamic programming to a realistic problem (cash dispensing).
- Provide a graphical interface for entering inputs and viewing results.
- Visualize the algorithm step by step, with an explanation for every cell.
- Explain the algorithm, analyse its time and space complexity, and test it with normal and boundary inputs.

## 2. Algorithm

### 2.1 Why dynamic programming

The problem has optimal substructure: if the best payout for an amount A ends with a note of value d, the remaining part must be the best payout for A - d. It also has overlapping subproblems: a plain recursion would recompute the same smaller amounts many times. Storing each answer in a table removes this repetition and reduces exponential time to polynomial time.

### 2.2 Recurrence

Let dp[a] be the fewest notes needed to pay exactly amount a.

```
dp[0] = 0
```

```
dp[a] = min over every note d <= a of ( dp[a - d] + 1 )
dp[a] = infinity if no note leads to a payable smaller amount
```

### 2.3 Steps

1. Set dp[0] = 0 and dp[a] = infinity for every other amount.
2. For each amount a from 1 to A, examine every note d with d <= a. If dp[a - d] + 1 is smaller than the current dp[a], store it and record choice[a] = d.
3. If dp[A] is still infinity, the amount cannot be paid with the given notes.
4. Traceback: start at A, take choice[A], subtract it, and repeat until 0. The notes collected form the payout.

When two notes give the same count, the larger note is preferred.

### 2.4 Pseudocode

```
MIN-NOTES(A, d[1..k])
    dp[0] = 0
    for a = 1 to A
        dp[a] = infinity
        for each note d[j] with d[j] <= a
            if dp[a - d[j]] + 1 < dp[a]
                dp[a] = dp[a - d[j]] + 1
                choice[a] = d[j]
    if dp[A] = infinity: report 'cannot be paid'
    else: walk back using choice[] to list the notes
```

### 2.5 Greedy versus dynamic programming

| Notes | Amount | Greedy payout | DP payout |
|---|---|---|---|
| 1, 6, 10 | 12 | 10 + 1 + 1 (3 notes) | 6 + 6 (2 notes) |
| 1, 10, 25 | 30 | 25 + 5 x 1 (6 notes) | 3 x 10 (3 notes) |
| 4, 5 | 8 | 5, then stuck with 3 left | 4 + 4 (2 notes) |

The standard ATM notes (500, 200, 100, 50, 10) work well with greedy, but this cannot be assumed for every denomination set. Dynamic programming is correct for all of them.

### 2.6 Optional extension: limited stock

A real ATM holds only a certain number of notes of each kind. The application accepts a stock for each denomination. Each stock s is split into chunks of size 1, 2, 4, ... plus a remainder (binary splitting), which turns the problem into a 0/1 knapsack over the chunks. Every count from 0 to s can be formed from these chunks.

## 3. Complexity Analysis

Let **A** be the amount and **k** the number of denominations.

| Part | Time | Space | Reasoning |
|---|---|---|---|
| Fill the DP table | O(A x k) | O(A) | A cells; each compares at most k notes. |
| Traceback | O(A) | O(k) | At most A notes (when note 1 exists); usually far fewer. |

| Part | Time | Space | Reasoning |
|---|---|---|---|
| Greedy comparison | O(k) | O(k) | One pass over the sorted denominations. |
| Plain recursion (not used) | Exponential | O(A) stack | Recomputes the same amounts repeatedly. |
| Limited stock | O(A x sum of log s_i) | O(A x chunks) | 0/1 knapsack over chunks; flags kept for traceback. |

**Overall:** O(A x k) time and O(A) space. Because the running time depends on the numeric value of A rather than the number of digits in it, this is a pseudo-polynomial algorithm, the same class as the 0/1 knapsack.

**Best case:** amount 0 or smaller than every note, so almost no work beyond the table setup.

**Worst case:** a large amount with many denominations, where every cell compares all k notes.

## 4. System Design

| File | Purpose |
|---|---|
| coin_core.py | The algorithm: input parsing and validation, DP with recorded steps, traceback, greedy comparison and the limited-stock solver. No GUI dependency. |
| atm_gui.py | The Tkinter interface that calls the core and draws the DP table. |
| test_coin.py | 29 automated unit tests covering correctness and boundary cases. |

### GUI features

- **Input panel:** amount, denominations, optional stock per note, and a drop-down of preset examples.
- **Controls:** Solve, Prev, Next, Auto-play with adjustable delay, Finish and Reset.
- **DP table:** one cell per amount. The cell being computed is gold, the cells it looks back at have an orange outline, the chosen one has a red outline, and the final payout path is blue.
- **Candidate panel:** for each cell, every note considered and the value dp[a - d] + 1 it would give.
- **Result panel:** the optimal payout, the greedy payout, a verdict, and the limited-stock payout when stock is entered.
- **Algorithm & complexity tab:** a built-in explanation of the recurrence and analysis.

### Input validation

The application rejects an empty amount or denomination list, text that is not a whole number, negative amounts, amounts above 1,000,000 (200,000 with stock limits), zero or negative denominations, duplicate denominations, and a stock list whose length does not match the denominations. For amounts above 300 the table is not drawn, but the result is still computed instantly.

## 5. GUI Screenshots

Figure 1 shows the application on start-up with the default ATM example loaded.

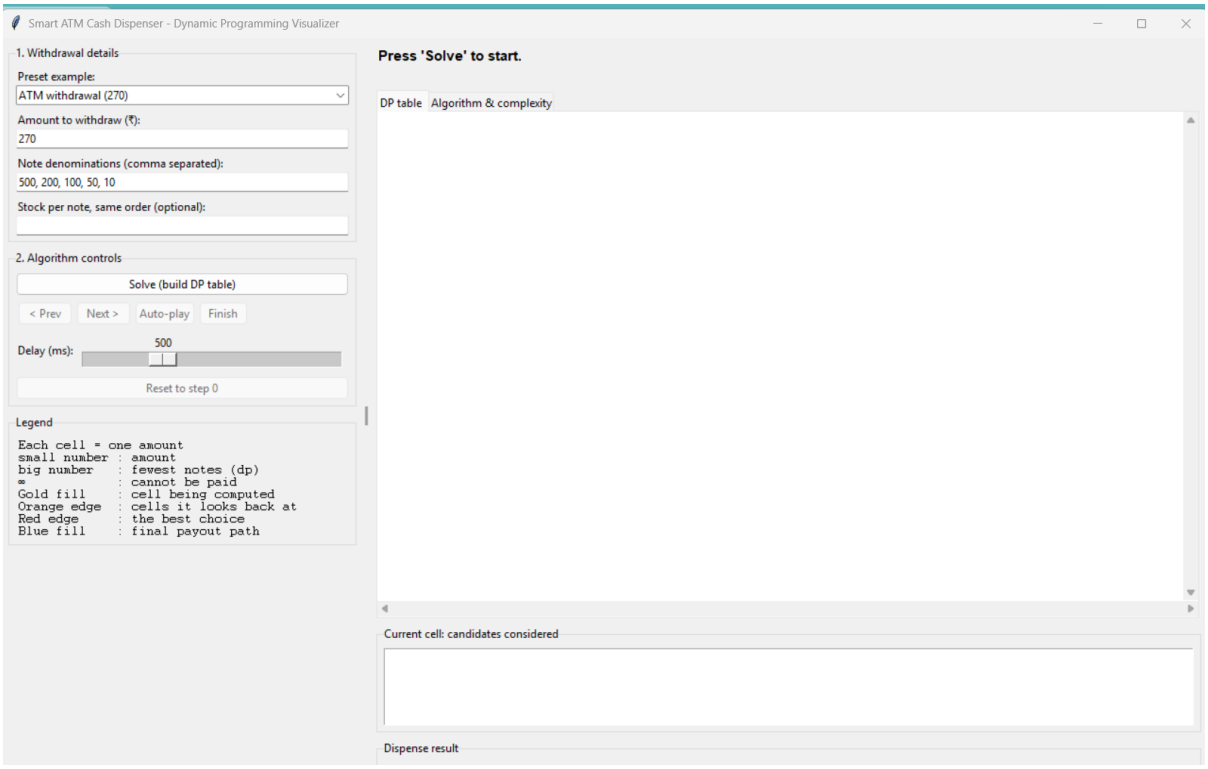



*Figure 1: Main window on start-up (amount 270, notes 500, 200, 100, 50, 10)*

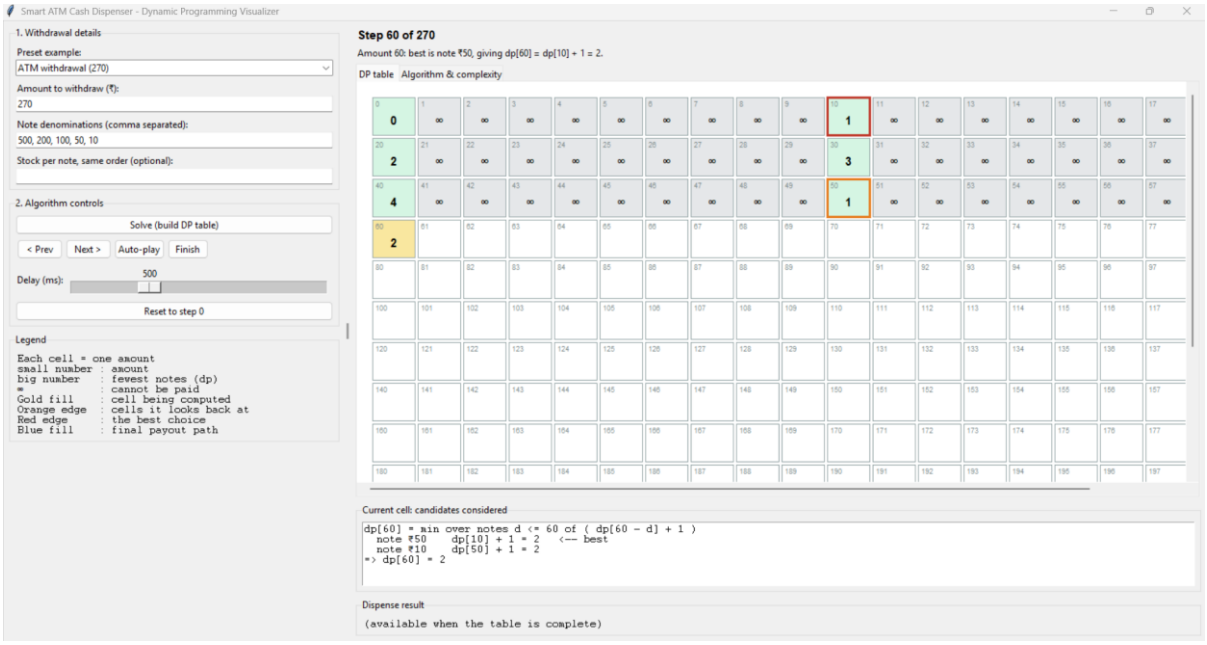



*Figure 2: DP table part-way through (for example step 60 of 270)*




*Figure 3: Completed table and result for amount 270 (200 + 50 + 10 + 10)*

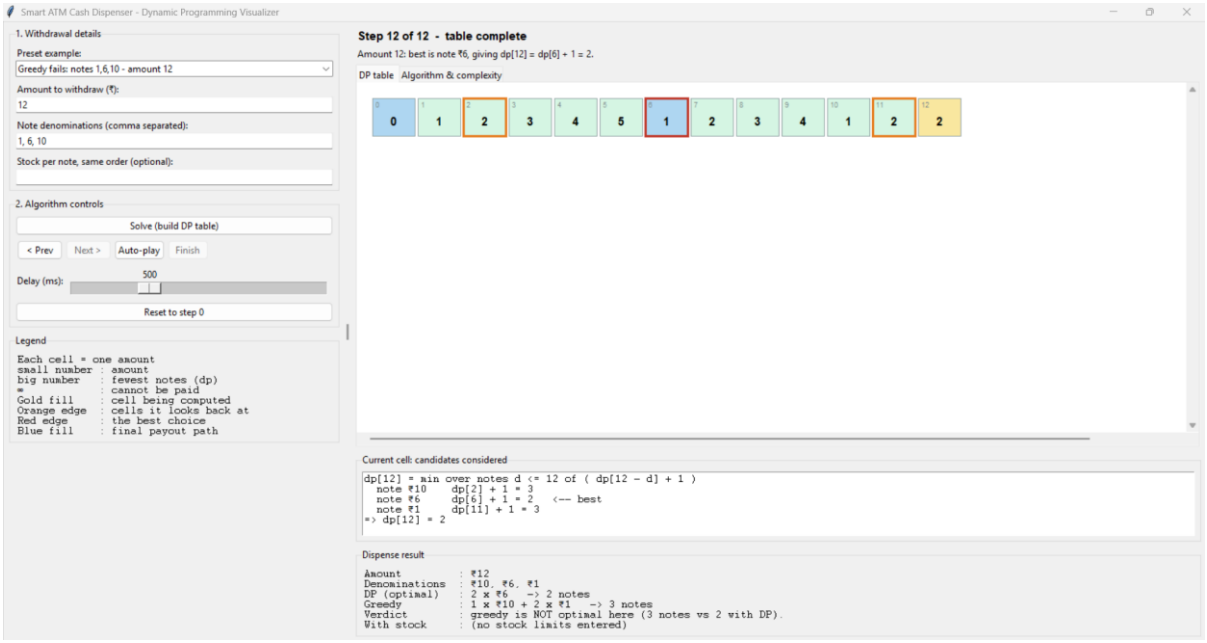



*Figure 4: Greedy is not optimal (3 notes) while DP uses 2 notes*

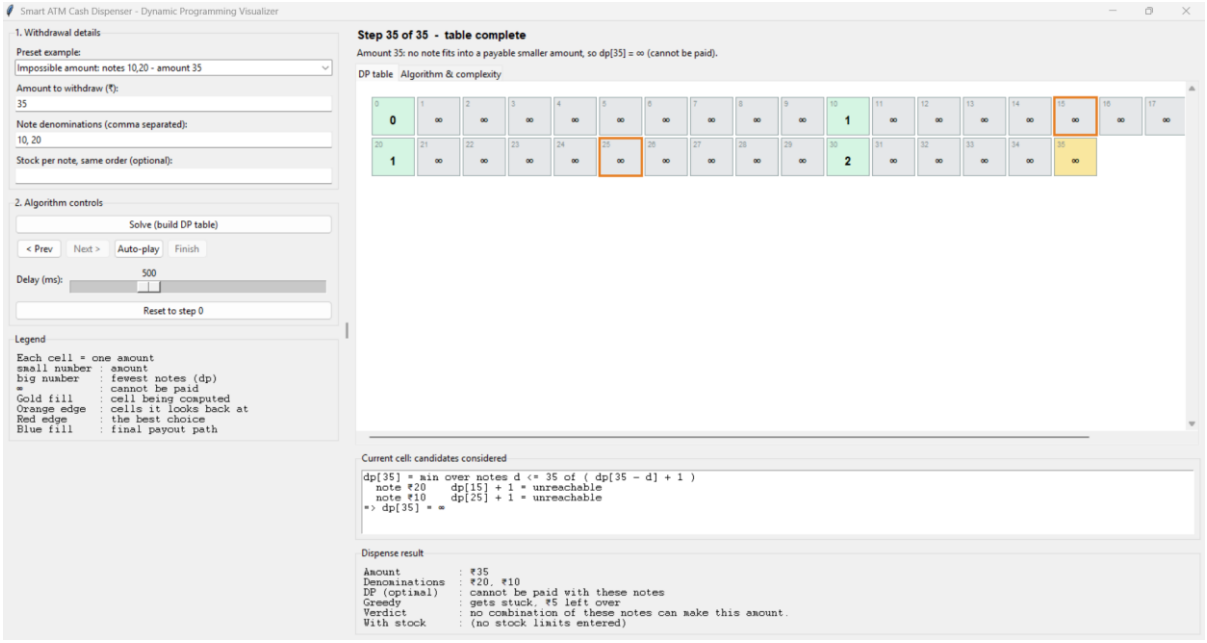



*Figure 5: Amount that cannot be paid*

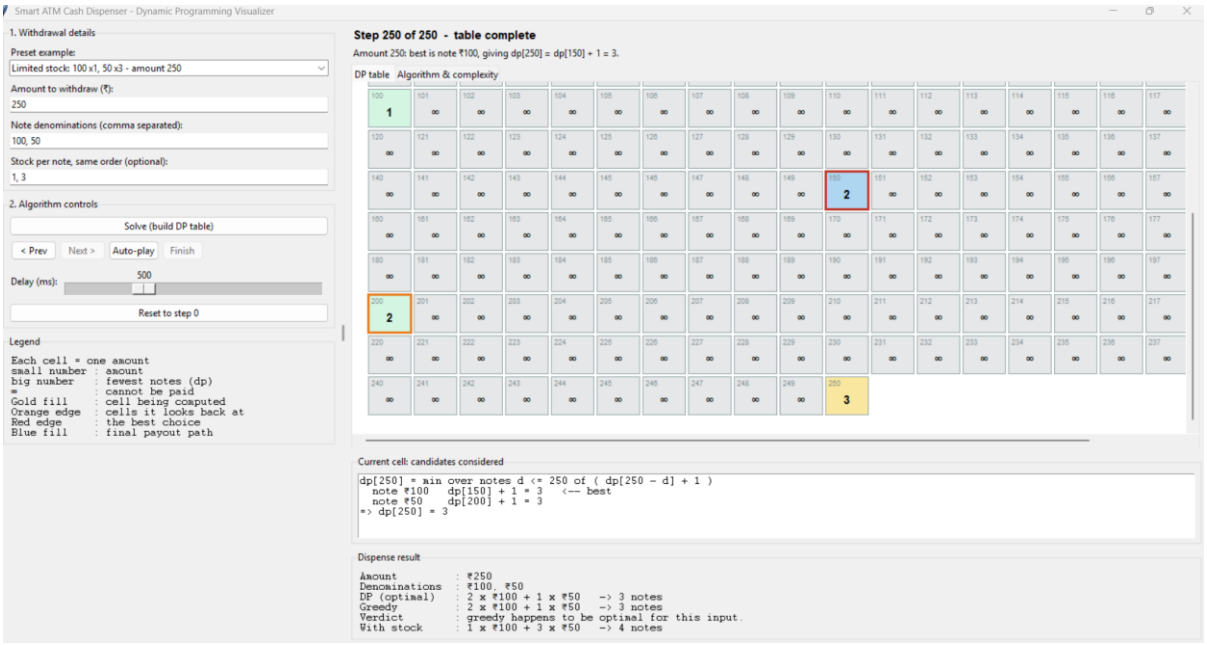



*Figure 6: Payout when stock is limited*

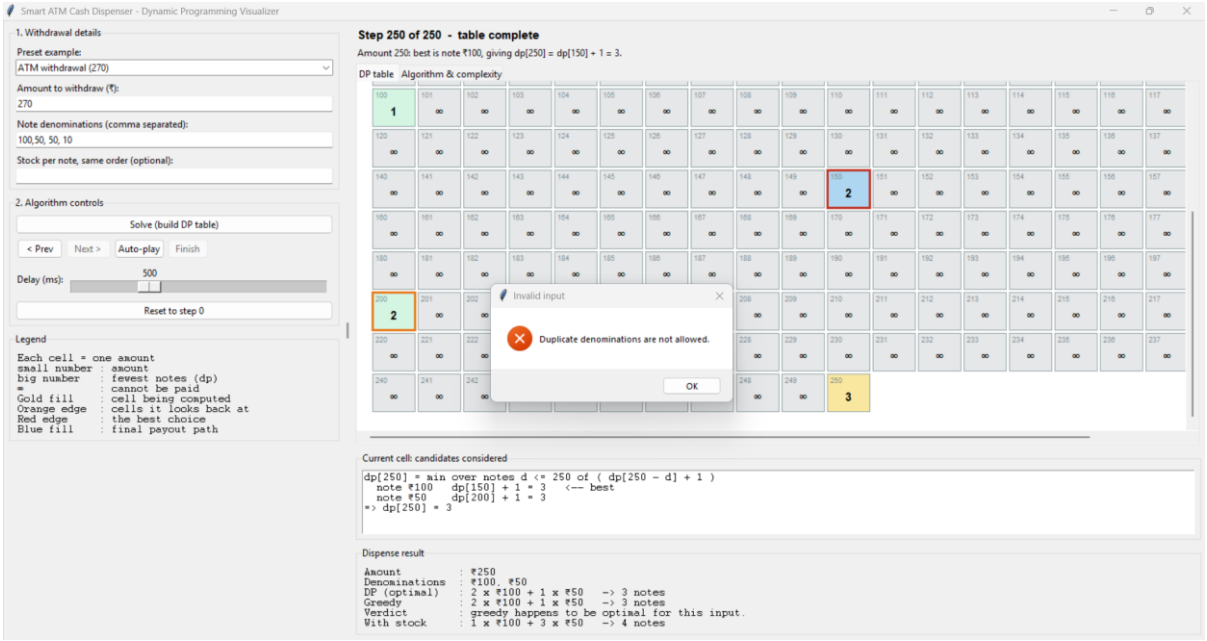



*Figure 7: Input validation message*

# 6. Testing and Results

The core engine was tested with 29 automated unit tests (`python -m unittest -v test_coin`). **All 29 tests passed.**

## 6.1 Boundary cases

| # | Test | Expected behaviour | Result |
|---|---|---|---|
| 1 | Amount 0 | 0 notes, empty payout | Pass |
| 2 | Amount smaller than the smallest note | Reported as cannot be paid | Pass |
| 3 | Impossible amount (35 with notes 10, 20) | No payout, empty path | Pass |
| 4 | Single denomination (50 and 55 with note 10) | 5 notes, then cannot be paid | Pass |
| 5 | Amount equals one note (500) | Exactly one note | Pass |
| 6 | Note 1 present (37 with notes 1, 5) | Always payable; 9 notes | Pass |
| 7 | Large amount (100,000) | 200 notes of 500, in well under 5 s | Pass |
| 8 | Maximum accepted amount (1,000,000) | 2,000 notes of 500 | Pass |
| 9 | Invalid denominations (empty, zero, negative, duplicate) | Rejected with an error | Pass |
| 10 | Invalid amount (negative, too large, decimal, boolean) | Rejected with an error | Pass |
| 11 | Input parsing (commas, spaces, semicolons; blank or non-numeric text) | Parsed or rejected correctly | Pass |

## 6.2 Correctness tests

| # | Test | Result |
|---|---|---|
| 12 | Typical withdrawal: 3770 gives 7 x 500 + 200 + 50 + 2 x 10 (11 notes) | Pass |

| # | Test | Result |
|---|---|---|
| 13 | Notes always add up to the amount and match the minimum count | Pass |
| 14 | Greedy fails: notes 1, 6, 10, amount 12 (3 notes vs 2) | Pass |
| 15 | Greedy fails: notes 1, 10, 25, amount 30 (6 notes vs 3) | Pass |
| 16 | Greedy cannot pay: notes 4, 5, amount 8, while DP pays it | Pass |
| 17 | Greedy equals DP for every multiple of 10 up to 2,000 with ATM notes | Pass |
| 18 | DP is never worse than greedy (300 random cases) | Pass |
| 19 | DP matches an independent breadth-first-search solver (300 random cases) | Pass |
| 20 | Recorded steps match the dp table for every amount | Pass |
| 21 | Traceback path runs from the amount down to 0 with (notes + 1) cells | Pass |
| 22 | Order of denominations in the input does not change the answer | Pass |

### 6.3 Limited-stock tests

| # | Test | Result |
|---|---|---|
| 23 | Stock forces more notes (1000 with 500 x1, 100 x10 gives 500 + 5 x 100) | Pass |
| 24 | Not enough stock (1000 with 500 x1, 100 x3) gives no payout | Pass |
| 25 | Very large stock gives the same count as the unlimited DP | Pass |
| 26 | A denomination with zero stock is never used | Pass |
| 27 | Amount 0 with stock | Pass |
| 28 | Mismatched stock length or negative stock is rejected | Pass |
| 29 | Matches exhaustive brute force on 200 random small cases, never exceeding any stock | Pass |

### 6.4 Sample runs

| Input | DP result | Greedy result |
|---|---|---|
| Amount 270; notes 500, 200, 100, 50, 10 | 200 + 50 + 10 + 10 (4 notes) | Same (4 notes) |
| Amount 12; notes 1, 6, 10 | 6 + 6 (2 notes) | 10 + 1 + 1 (3 notes) |
| Amount 30; notes 1, 10, 25 | 3 x 10 (3 notes) | 25 + 5 x 1 (6 notes) |
| Amount 8; notes 4, 5 | 4 + 4 (2 notes) | Stuck: 3 left over |
| Amount 35; notes 10, 20 | Cannot be paid | Stuck: 5 left over |
| Amount 3770; notes 500, 200, 100, 50, 10 | 7 x 500 + 200 + 50 + 2 x 10 (11 notes) | Same (11 notes) |
| Amount 250; notes 100, 50; stock 1 and 3 | With stock: 100 + 3 x 50 (4 notes); unlimited: 2 x 100 + 50 (3 notes) | 2 x 100 + 50 (3 notes, ignores stock) |

**Observation:** for ordinary ATM denominations greedy gives the same note count as DP, which is why it is widely used. The difference appears with unusual denomination sets and with limited stock, where DP stays correct and greedy does not.

## 7. Conclusion

The project applies dynamic programming to minimum-note cash dispensing and makes the table-filling process visible step by step. The solution runs in O(A x k) time and O(A) space, always returns an optimal payout, and correctly reports amounts that cannot be paid. The built-in greedy comparison shows why dynamic programming is needed. Possible extensions are counting the number of distinct ways to pay an amount, supporting several currencies, and balancing note usage so that no denomination runs out first.

## 8. How to Run

```
python atm_gui.py                   # start the application
python -m unittest -v test_coin     # run the tests
```

Python 3.8 or later with Tkinter is required. No external libraries are needed.